Introduction:

In many games, one player's gain is another's loss, which creates a range of incentives for self-interest and a notion of collaboration to avoid retaliation. A prominent example of this early on in the semester was watching the two men on "Golden Balls" try multiple negotiation strategies to convince each other to split; beyond the entertainment value, the most prominent part of the clip is the number of times the two players go back and forth before finally locking in their choices. As such, we wanted to build on top of the Ultimatum Game with slight variations for this project. At a high level, the Ultimatum Game requires two players, one who determines an offer of how to divide a sum of points and another who chooses to accept; the interesting part is that if the responder chooses not to accept, both players lose, which calls back to the delicate balance of self-interest in getting a larger split and self-preservation in not provoking the other player to spite you.

We can build simple agents that emulate strategies such as "always reject uneven splits" or "always accept," and we can also build reinforcement learning models to analyze past games. LLMs become particularly useful because not only can they select a split or reply with a boolean if we give them structured input models, but they can generate text quickly as well. Instead of coming up with methods to show intent in a purely heuristic or reinforcement learning model, we can shift our focus to building interesting prompts and track where the agents deviate from our preconceived notions of the game. Unlike a heuristic-based model where we can make something greedy by increasing an integer in a configuration, using LLMs allows us to create natural language strategies and even invoke the personalities of real people and characters alike.

As important as the system prompt is, another key consideration of ours is how we allow agents to communicate with one another. In the Golden Balls, the players were able to go back and forth several times and at least it seemed like the way they were talking to each other made a difference. We would like to take interesting strategies and characters and try to empirically understand if the way players communicate can make a difference in game outcomes.

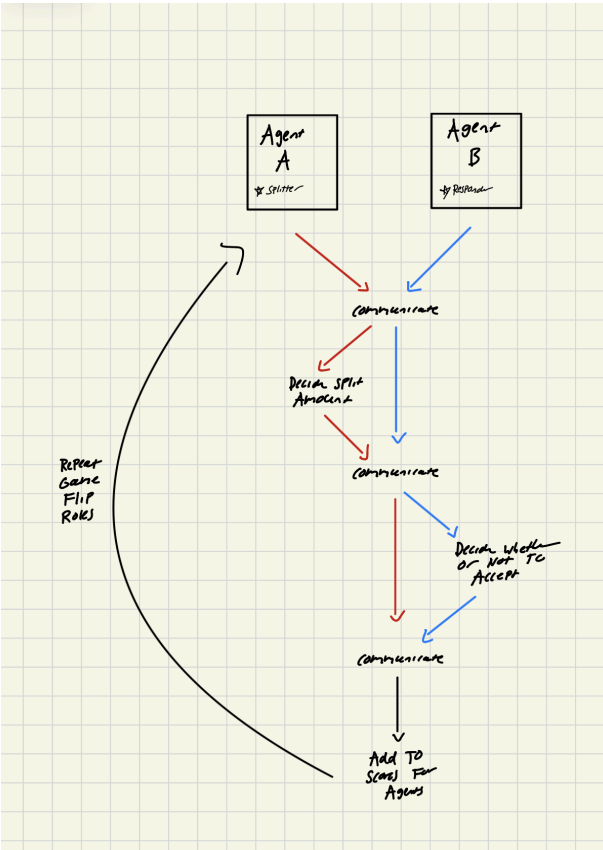
As a result, the utilization of LLMs here joins our interest in repeated games with the ability to explore how LLMs can negotiate, coerce, or otherwise work together to find endgame solutions with varying levels of communication and system prompts. Our main research question is how increasing levels of communication affect game outcomes in terms of total utility across different case studies of system prompts.

System Architecture:

In our project we created a mix of heuristic-based agents and LLM agents to test our game framework for the Ultimatum Game. We did not use the heuristic-based agents for anything meaningful other than some very basic tests early on to verify our game framework, which is written in Python and packaged using uv. We used the Tufts LLM Proxy for a large part of our data collection and needed to rely on the OpenAI SDK to handle some cases where the Tufts LLM Proxy's content filter prevented us from running experiments; these were both called from the Python game framework. To keep results consistent, we used GPT-5-mini to balance performance speed and the improved reasoning that OpenAI claimed came with the jump from the 4 to the 5 series.

Within our framework, to build a LLM enabled agent, you first create a system prompt in the constants.py file. You then create a game configuration in game_runner.py which specifies the communication level, number of rounds, and the system prompts of the agents involved. With the runner created, ensure that the correct API key is set in the .env file and you can run your own simulations.

Within each simulation, the previous freetext chatter, splits, and acceptances are all stored in the context window of the LLM and we reset this per session. This allows the agents to learn and adapt to each other, while allowing us to cycle through these per simulation to avoid any spillover.



In the ultimatum game, there are two roles: the splitter, who chooses what the split will be, and the responder, who chooses whether the offer will be accepted or if both players will lose. Every round the agents will trade roles to ensure they have a fair experience and we account for potential retaliation against bad behaviors. Each agent makes one choice per round, both of which are bound by JSON schemas that force a boolean for the responder and a float between 0 and 1 for the splitter.

In a given game, agents are assigned system prompts which are intended to represent either articulated strategies or personas for the agents to try and emulate. We intentionally build out a framework to allow the agents to be blank canvases on which we can apply a system prompt with one variable assignment and create unique case studies.

In a given round the agents have four actions: choose split, choose to accept an offer, listen to freetext chatter, and generate freetext chatter. Within the game framework, agents are limited to which action they can take at a given point in time because we rigidly invoke specific functions, such as when to generate freetext, as part of the game framework. We do not maintain memory within the game framework, instead we directly rely on using an extended context window and rely on the LLM backend to gracefully keep the session intact, allowing for us to get a notion of memory.

Each communication window in the vertical column to the left represents four calls to the LLM API proxy: one for an agent to speak and one for the other agent to store that in their context window, effectively modeling listening, and then applying this the other direction. We limited communication to 2-3 sentences per generation to avoid clogging the context windows up given the limited resources on the Tufts LLM proxy. As part of our core research question, we toggled specific intervals on or off and refer to the three time windows to speak as:

- Pregame: before the split is given
- Midgame: after the split is given and before the accept decision is made
- Postgame: after the acceptance decision is made effectively allowing agents to say “nice job” or “watch your back” depending on the outcome.

The game framework itself keeps track of rounds played in a particular iteration, the individual round results with chat logs, states via session tokens for LLM agents; this allows us to perform analysis on the offers and acceptance rates to understand how the total utility per agent played out. We then store these objects in cold storage by dumping them to JSON once a game is completed and are able to quickly read these objects in to do basic analysis using Python.

LLM Integration:

We used LLMs to act as decision-makers during the Ultimatum Game, where they faced off against other agents. Our game framework serves as a structured protocol for the LLMs to interact with the game environment, but the decisions and game dialogue are as freely generated by the agents as possible. More than anything, we saw the benefit of using a LLM here to be that we could articulate ideas and see how the agents would play them out instead of having to explicitly write policies or basic heuristic-driven agents. For this purpose, we didn't analyze any interaction between LLMs and heuristic-driven agents to focus on using the flexibility a system prompt gives us. To give a picture of what is ahead, our case studies ranged from writing a basic strategy that we might articulate in a game theory lecture to invoking characters from pop culture. As we increased communication volume in the game configurations, it was particularly interesting to see the LLMs deliver on the promise of assuming the roles they are given and even using the catchphrases of the characters we assigned in their gameplay.

Thus, the primary role of the agents in our multi-agent system is to play the two key roles in the game and negotiate, gaslight, or otherwise communicate with the other agent to generate utility by scoring points. This is not a distributed decision-making project, it is a focused series of case studies where the LLMs serve as the main personalities and driven forces for the decisions made.

For the technical implementation, we used the Tufts Proxy API to process our LLMs. Throughout the game, we allowed agents to interact with each other, utilizing the API during every exchange. The API is called to make the split or accept decision and there are up to three different junctures where one agent speaks and the other listens before returning the favor, meaning that our cost function jumps significantly per round as we try to increase the amount of communication. We largely got around these costs by starting the simulations early and trying our best to spread the load out across days to take advantage of the given quota.

During these interactions, however, we discovered a technical challenge: the Tufts Proxy had a content filter that sometimes prevented any output, which hindered our progress. We confirmed that there was not anything particularly heinous; it seemed that our agents would use phrases like "rejecting only hurts you" or "I'll reject any offer under 30% to show I won't be exploited" when we allowed them to talk at all three game stages. The mere mention of harm or exploitation in the outputs which are then passed into the other agent's content window may have well been enough to trip this filter. To address this, we switched to using the exact same GPT-5-mini model, which does not have the content filter constraint of the Tufts Proxy. After making the switch, we received complete output for the games that were previously failing.

Experiments and Results:

Within each case study, we kept the following factors constant:

- Agents did not know the game length
- Agents had memory based per-iteration; that is, each N round game was contained to one "memory"
- For each case study, the number of rounds was kept consistent, though cross-study volumes varied
- Using GPT-5-mini via Tufts' BridgeLLM proxy and the OpenAI SDK
- The prompts giving turn-based instructions remained the same, making the primary difference across case studies simply the system prompt

Our dependent variable per case study was the communication levels that agents were allowed to use each game:

- All communication means each agent speaks before the split is made, before the offer is accepted or denied, and after the round outcome has been determined

- Midgame communication means each agent speaks before the offer is accepted only, not at the start or end of the round
- No communication means none of the three communication points above are allowed, the agents can only choose a split and respond accept or deny

In each case study, we first focused on the changes in utility as communication changed, but interesting trends among offered splits and specific logs emerged in some studies while others were not as interesting. We commonly refer to the "acceptance rate" of game configurations; recall that since the total utility of the game only increases if an acceptance occurs, this effectively counts the sum of utility over the number of rounds.

Each case study was then analyzed for interesting trends in total utility which allows us to understand how well the agents worked together among other factors. **In all cases where we introduce an "all communications" scenario, we needed to use the OpenAI SDK due to aforementioned concerns about content filtering. We tried our best to do reasonable amounts of experiments over the span of two weeks to ensure we were not unfairly burning through quota. We would like to emphasize this notice in the beginning of this section, but please recall that this was constant across all three of our case studies for the all communication configurations.**

Please note that all prompts used can be found in the constants.py file attached to our project submission.

Case Study One: Varied Communication Levels with Two "Strategy-Based" Agents

We chose to start by exploring agents when given abstract strategies. We described attitudes, but did not give examples of real world people or fictional characters in an attempt to see game outcomes when a strategy was effectively spelled out for them. To run this experiment, we ran 40 iterations of 10 rounds of the Ultimatum Game with the following configuration held constant for each agent:

- A "greedy-to-rational" system prompt that emphasized starting greedy, but nudged the agent to be mindful that both agents lose if they do not eventually cooperate.

The clearest takeaway from this case study was that we observed significant shifts in acceptance rates, and thus total utility, as communication protocols increased. In the "All comms" configuration, we saw 10x the utility because the players were able to make deals with each other by talking out their initial biases to find common ground. Subsequently, we see a slight decrease in the average split in these two configurations. What was more unintuitive was how the splits for the midgame only configuration had the highest averages of accepted splits; it is possible that while the offers could not be shaped in this configuration, some kind of deal is consistently reached across players to yield more rounds that reach agreement.

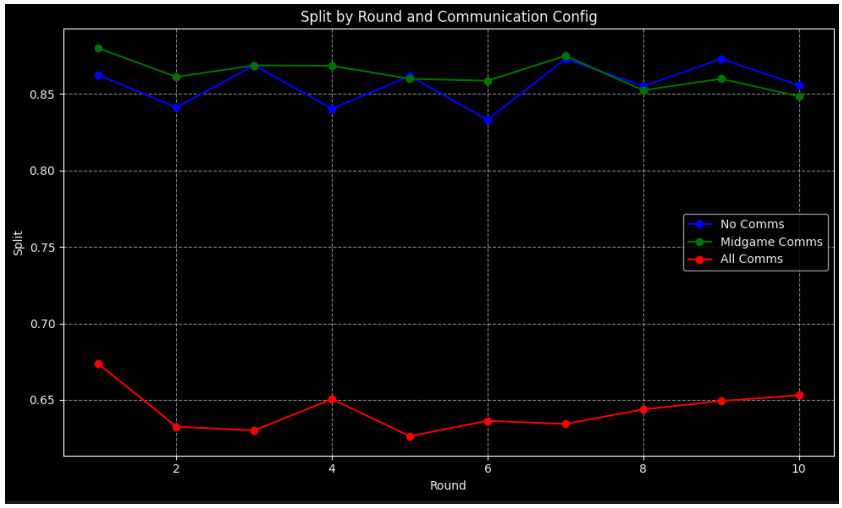
Average Values of Accepted Splits
Mean and standard deviation by communication level

Configuration	Mean	Stdev
No comms	0.72	0.18
Midgame Comms	0.78	0.16
All comms	0.62	0.07

Acceptance Rate per Game Configuration
Time Traveler Character Agents

Configuration	Mean
No comms	0.07
Midgame Comms	0.21
All comms	0.79

Another emerging trend in this game configuration is what we call a splitter's bias. Since the range of accepted offers primarily stays above 60% at all communication levels, we believe the agents maintain a "splitter's bias" which varies based on the communication level of the game.



As we can see in the average splits per round, the red line representing all communications stays well above 50%, meaning the agents still maintain a notion of their original strategies even though they can heavily communicate. For the game configurations with less communication, we see sustained greedy splits which are more extreme. Overall, our strategy-driven case study highlights that the agents do not totally abandon their system prompt when they can talk with each other freely and that increased communication levels yield higher total utility than not.

Case Study Two: Testing Persona-Based Game Configurations with Varied Communications

After describing strategies to the LLMs, we wanted to describe abstract characters or positions to see if we would see similar outcomes with a different style of system prompt. To run this experiment, we ran 20 iterations of 5 rounds each; since we experimented with many personas to find interesting ones, we needed to reduce resource consumption a bit.

Our two personas were:

- A FTC regulator with a sense of justice and disdain for greed which was loosely based off of Lina Khan, but we specifically left her name out of the prompt to keep this a persona.
- A greedy banker who works 80-100-hour work weeks with a short temper and a brash personality

We chose these because we believed that these two prompts would be inherently standoffish and intentionally encoded that into the personas by referring to disdain for some traits held by the other.

Acceptance Rate per Game Configuration

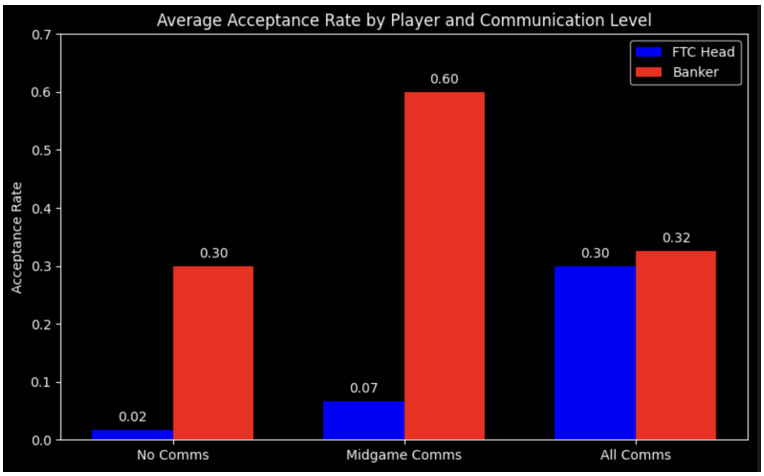
Time Traveler Character Agents

Configuration	Mean	Count
No comms	0.13	100
Midgame Comms	0.28	100
All comms	0.31	100

Unlike the strategy-based game configuration, the “all communications” portion of this case study was only 2.4 times larger than the no communications batch. Furthermore, the midgame communication was almost indistinguishable from the all communications format. At first, we were confused with how there could be such a small jump, but inspecting our logs revealed incompatible personality types between our agents and a clear barrier to compromise. Here is just a snippet of their banter highlighting incompatibility:

Banker: Spare me the sanctimony I trade in cold numbers, not sermonizing. Offer me at least 60% and we'll talk; anything less and I walk and take nothing

FTC Regulator: I don't tolerate slimy greed. I fight corporations that rig the system, and I expect the same decency here. I'm proposing a fair split; accept it if you value fairness, or try to squeeze more and you'll walk away with nothing



Given these chat logs, it should be no surprise that in game formats with limited communication, the FTC chair rarely took the unfair offer from the banker. We still saw some notions of increasing communication levels leading to increased offer acceptance and thus higher utility; however, what was most interesting about these agents is that the FTC agent took a sense of justice to heart and they only changed their acceptance behavior when they could impose it on the banker through dialogue.

Case Study Three: Testing Character-Based Game Configurations with Varied Communications

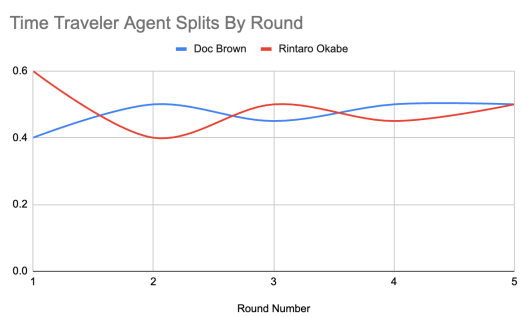
With strategies and personas out of the way, we also wanted to take the opportunity to let agents play roles of specific characters; we landed on fictitious characters largely to see how the agents would act, putting these characters in roles they wouldn't in their source media since our other case studies were more closely grounded in reality. Once more we used a 5 round and 20 iteration per communication level layout after going through other less interesting characters. Our two agents were encouraged to acknowledge themselves and their catchphrases and were:

- Doc Brown from Back to the Future
- Okabe Rintaro from Steins; Gate.

Acceptance Rate per Game Configuration

Time Traveler Character Agents

Configuration	Mean
No comms	0.82
Midgame Comms	0.87
All Comms	1.00



Our time travelers had a very high level of agreeableness from the start and the total utility slightly increased as they got to talk to each other more. With this game, we also found that the agents were finally able to converge on 50/50 splits; given our previous cases, this only highlights how malleable LLMs can be given different system prompts. Throughout the game, the Doc Brown agent believed in giving a more generous share, while the Okabe agent believed that he was owed a bigger share because of the burden from his past, a reference to the source material.

Overall, our character-based case study continued our trend of increased communications increasing total utility and highlights that the LLMs can not only implement strategies to play out the game, but can encode a strategy based on a specific character.

Discussion:

Benefits and Limitations of LLM-Enabled Multi-Agent Systems

A clear benefit of using LLMs in multi-agent systems is that it can be used as a way to quickly prototype or reason about adaptive learning tasks. Instead of needing Q-learning or encoding the problem space meticulously, individuals are able to use natural language and pass a single session ID into their API calls and play out a game framework. Of course, this is a simplification, but allowing researchers to be more nimble and iterate through ideas faster is a real benefit we took advantage of in this project.

Similarly, we were able to experiment and get creative with the personas used in our Ultimatum Game sessions. It would have been much harder to make a Doc Brown agent in a purely quantitative sense and it is more likely than not that we would not have been able to get that far if that was our project. Even with the greedy-to-rational strategy, we did not need to set ranges that were reasonable as the game went on and instead were able to explore how the agents would come up with those boundaries within the scope of a game. Of course, there are some natural limitations in that even the "why" that a LLM provides may not be a sufficient technical explanation for rigorous academic or research tasks. In no way do we claim that a LLM can perfectly do reinforcement learning, but we do believe that our agents handled context windows and adaptation reasonably well while primarily using GPT-5-Mini as the decision engine in our simulator.

A significant limitation of using LLMs in multi-agent systems can be API latency. In a situation where speed is critical, it may not be wise to use an LLM. As we observed above, the no communication games were very quick, largely because we provided fixed input JSON schemas and the outputs were bounded floats or booleans. As we increased the amount of communication, the overhead of each following API call was apparent. In our brief interaction with OpenAI's API compared to the Tufts proxy, we saw some improvement in the speed of individual calls; however, this was for games with 16 calls to OpenAI. For a system which requires many calls with few tokens, network latency becomes a real consideration. We were fortunate that we had weeks to collect data to spread the load and debug these kinds of issues, so limitations around token usage were less relevant than in other projects.

Challenges with our Project and LLM-Enabled MAS

Our biggest unexpected challenge was surely running into the content filter on the Tufts BridgeLLM proxy. As we mentioned above, our chat logs did not contain vulgarity and the midgame communication-based games ran into no obvious issues in this regard. This required us to route our calls to OpenAI for the same model and context window, which should function the same as going through the Tufts proxy except for the content filter. While we did try to be respectful of token usage and the scope of the assignment to get this done, it does bring up a broader conversation about unchecked guard rails even if our use case is trite compared to military or medical applications of LLMs.

While we were able to verify that our agents were not doing anything reprehensible, this highlights a broader challenge with LLMs: safety and responsible use. While it is great to iterate quickly and explore new ideas in ways that were not previously available, that leaves many vulnerable. The scope of this project was to use any means necessary, ranging from honesty to lies or threats, and see what the LLMs would lean toward, but there are many projects where any means necessary is not the line.

Another challenge was finding highly useful trends in the freetext logs. Even when agents start agreeing and accepting, it seems that none of the agents really change their tone. For the most part, the logs before and after

inflection points in games tended to simply include more agreeable splits, not some significant strategic play from one agent to another. The logs were certainly helpful for debugging, verifying that certain past events remain in the memory of the agents, but they had limited analytic value. Even if we were to do basic NLP on them, it is more likely than not that the sentiment remains primarily flat throughout the games since numbers changing amidst threats and negotiation are not interesting sentiment shifts. As a result, we leaned heavily into the game results from a quantitative side with the logs largely serving a demo purpose instead of being the main point of analysis, despite taking up the vast majority of the token generation for this project.

Future Work

The most obvious future work is a function of our largest problem: we would like to run these simulations again using only one proxy to ensure that there are no confounding effects which come from using the same agent with two different proxies. While there is not an obvious reason to believe proxy changes would greatly affect outcomes, it does prevent us from being fully confident in our results.

With more time and resources, it may also be worth exploring more granular configurations of communications and expanded game sizes. Of course, we selected midgame communications since it was the closest action to both decision points; however, it could be just as valid to explore pregame or postgame communications only; and that is before we consider all permutations. Similarly, expanding game length to see if any notions of cooperation eventually break down or revert may be worthwhile, especially with adversarial agents similar to our greedy or banker prompts.

One final consideration which greatly expands the scope of our project would be to apply the same principles of UltimatumLLM, strategy or persona encoding and fixed natural language communication protocols, to debate preparation or some similar real world application. All it would realistically take is finding a way to encode the responses of the human who is practicing for the debate and ensuring that the pacing of responses aligns with that. Since there is ample training data that LLMs are likely trained in on policy, this could be a lightweight way to improve one's debate skills.

Conclusion:

Overall, UltimatumLLM serves as a jumping off point into encoding personas into strategies for LLM-based agents to play games and explicitly controlling the communication flows that they use. We have cursory evidence that the more that agents speak over the course of the game, the higher total utility they'll have because they will be making acceptance splits more often. Even agents with greedy or somewhat harsh strategies do show some signs of warming up to collaboration with high levels of communication; agents which have a higher baseline level of agreeableness can still see results, though of course they have much less room to grow.

At the same time, we saw clear differences across the three sets of prompts we came up with and that even within a game, the acceptance rates can greatly differ between the two players, especially if their strategies are not the same. We believe a reasonable follow up project could be to explore the difference between playing out a strategy and taking on the persona of a human being, fictional or not. It could be the case that if we shape the LLM's context so that it is trying to act as if it were that person versus being some automaton which plays out a strategy.

If you would like to experiment with writing your own prompt or character, spinning up some agents, and letting them compete to answer some of these broader questions, you are welcome to clone this Git repository and explore.